encapsulate the binding targets for the username and email. You need to add properties for the first name, last name, and address, so add the following lines of code to the `InputModel` class declared within the `RegisterModel`.

Listing 9.17 Adding properties for the additional registration data you want to capture

```
[Required]
[Display(Name ="First Name")]
public string FirstName { get; set; }
[Required]
[Display(Name = "Last Name")]
public string LastName { get; set; }
[Required]
public string Address { get; set; }
```

At this stage, if you changed the `RequiredLength` value when configuring the password options, you should adjust the `MinimumLength` property value on the `StringLength` attribute that has been applied to the `Password` property of the `InputModel` accordingly.

Next you need to assign the bound values of your new properties to the user. This happens in the `OnPostAsync` method, which contains a lot of code. However, at this stage, you should only be interested in the bit where the user is created if `ModelState` is valid.

Listing 9.18 Creating the user if `ModelState` is valid

```
if (ModelState.IsValid)
{
    var user = CreateUser();                                          ❶

    await _userStore.SetUserNameAsync(user, Input.Email,              ❷
    ➥ CancellationToken.None);                                       ❷
    await _emailStore.SetEmailAsync(user, Input.Email,                ❸
    ➥ CancellationToken.None);                                       ❸
    var result = await _userManager.CreateAsync(user,                 ❹
    ➥ Input.Password);                                               ❹
```

❶ An instance of the user is created by the private CreateUser method at the bottom of the file.